

GUÍA DE DEFENSA ORAL

Trabajo Práctico Integrador — Etapa 1

Base de Datos II — Grupo 144

Damian Tristant | Ximena Maribel Sosa

● Ximena

● Damian

ESTRUCTURA DE LA DEFENSA (15 minutos)

La calificación es individual. Cada integrante debe demostrar que comprende las decisiones técnicas del proyecto. La división propuesta asegura que ambos hablen en profundidad sobre diferentes aspectos.

Bloque → **Responsable** → **Tiempo aprox.**

- **Apertura y mejoras incorporadas** → **Ximena** (1 min)
- **Descripción del negocio y por qué NoSQL** → **Ximena** (1 min)
- **Mostrar y explicar las 4 colecciones en Compass** → **Damian** (3 min)
- **Por qué embedding** → **Ximena** (2 min)
- **Por qué linking / referencia** → **Damian** (2 min)
- **Baja lógica** → **Ximena** (1 min)
- **Índices (concepto y por qué no los usamos)** → **Damian** (1.5 min)
- **Validaciones (concepto y cómo se usarían)** → **Damian** (1.5 min)
- **Infraestructura Atlas y acceso del profesor** → **Damian** (1 min)
- **Preguntas del docente** → **Ambos** (variable)

BLOQUE 1 — Apertura y mejoras incorporadas

●● Ximena

"Buenas, somos Damian Tristant y Ximena Sosa, Grupo 144. Antes de comenzar queremos aclarar que incorporamos las mejoras señaladas en la retroalimentación. El modelo original describía un problema de trazabilidad de materiales, pero las colecciones no lo resolvían. Por eso agregamos una cuarta colección llamada materiales, incorporamos el campo materiales_requeridos dentro de productos, y agregamos insumos_utilizados dentro de cada ítem de las órdenes de trabajo. Ahora el modelo sí responde al problema planteado."

BLOQUE 2 — Descripción del negocio y por qué NoSQL

 Ximena

"El taller 'Muebles del Bosque' no tenía forma de saber qué materiales se consumieron en cada mueble, qué cliente hizo cada pedido, ni en qué estado estaba cada fabricación. Diseñamos una base de datos NoSQL en MongoDB con cuatro colecciones: clientes, productos, materiales y ordenes_trabajo."

"Elegimos MongoDB porque es una base de datos documental que trabaja con formato JSON. El principio central del modelado NoSQL que vimos en clase es que los datos que se acceden juntos deben almacenarse juntos. En nuestro caso, una orden siempre se consulta con sus items y sus materiales, por lo que tiene sentido tenerlos en el mismo documento. A diferencia de una base relacional como MySQL, no necesitamos esquemas rígidos ni joins entre tablas para obtener la información completa de una orden."

BLOQUE 3 — Mostrar y explicar las 4 colecciones

 Damian

Colección: clientes

"La colección clientes tiene nombre, email, un subdocumento embebido llamado contacto con teléfono y dirección, y el campo activo para la baja lógica. En MongoDB los subdocumentos son objetos anidados dentro del documento principal, representados como pares clave-valor en formato JSON."

Colección: productos

"La colección productos tiene nombre, categoría, precio base, y un array llamado materiales_requeridos. Un array en MongoDB es una lista ordenada de elementos. En este caso, cada elemento es un objeto con nombre del material, cantidad y unidad de medida. Esto representa qué insumos necesita cada mueble para ser fabricado."

Colección: materiales

"La colección materiales es el catálogo de insumos del taller. Tiene nombre, unidad de medida, stock actual, proveedor y precio unitario. Existe como colección independiente porque el stock se actualiza solo, sin depender de las órdenes."

Colección: ordenes_trabajo

"La colección ordenes_trabajo es la colección central del sistema. Tiene número de orden, una referencia al cliente mediante cliente_id, fechas de inicio y entrega estimada, un array de items donde cada item tiene el nombre del mueble, precio acordado y los insumos utilizados, el estado de la orden y el total."

BLOQUE 4 — Por qué embedding



"En MongoDB existen dos formas de relacionar datos: embedding y linking. Embedding significa guardar los datos relacionados dentro del mismo documento. Según lo visto en clase, se usa cuando los datos se consultan siempre juntos, cuando la relación es uno a uno o uno a pocos, y cuando los datos no crecen indefinidamente."

"Nosotros usamos embedding en tres casos. Primero, el contacto del cliente está embebido porque es una relación uno a uno de alta cohesión: la dirección y el teléfono no tienen sentido fuera del contexto del cliente y siempre se consultan juntos. No tiene sentido una colección separada solo para guardar direcciones."

"Segundo, materiales_requeridos está embebido dentro de productos porque los materiales son parte constitutiva del diseño del mueble. Cuando consultamos un producto siempre necesitamos saber qué materiales lleva. Siempre se acceden juntos."

"Tercero, y más importante, insumos_utilizados está embebido dentro de cada ítem de la orden. Esta decisión resuelve la trazabilidad histórica: si mañana el precio de la madera sube, o cambia su nombre en el catálogo, la orden histórica no se modifica porque los valores quedaron congelados en el momento de la fabricación. Esto protege los reportes de facturación pasados."

BLOQUE 5 — Por qué linking / referencia



"Linking significa guardar solo el ID del documento relacionado en lugar de copiar todos sus datos. Según lo visto en clase, se usa cuando la relación es uno a muchos con crecimiento potencial no acotado, cuando los datos se consultan por separado, o cuando muchos documentos comparten la misma información."

"Nosotros usamos linking en el campo cliente_id dentro de ordenes_trabajo. La relación es uno a muchos: un cliente puede tener cientos de órdenes. Si embebíamos todos los datos del cliente dentro de cada orden, duplicaríamos su nombre, email, teléfono y dirección en cada documento, generando redundancia masiva."

"Además, si el cliente actualiza su teléfono o email, con linking actualizamos un solo documento en la colección clientes y todas sus órdenes quedan automáticamente consistentes. Si hubiéramos embebido, tendríamos que modificar cada orden individualmente."

"También hay linking entre productos y materiales. La colección materiales tiene ciclo de vida propio: su stock se actualiza independientemente de las órdenes y es compartida por múltiples productos."

BLOQUE 6 — Baja lógica

 **Ximena**

"El negocio estableció que ningún registro puede eliminarse físicamente del sistema, por normativas de auditoría interna y resguardo de datos históricos de facturación. Para implementar esto incorporamos un campo booleano activo en cada documento de todas las colecciones."

"Cuando se da de baja un cliente, un producto o una orden, en lugar de ejecutar un deleteOne que lo elimina para siempre, ejecutamos un updateOne con \$set activo: false. El documento sigue existiendo pero el sistema lo trata como inactivo."

"Esto se puede ver en la colección clientes donde tenemos documentos con activo: false, como Juan Carlos Pérez y Ana Martínez. Sus órdenes históricas siguen existiendo y pueden consultarse para auditoría sin afectar la operación normal del taller."

BLOQUE 7 — Índices

 **Damian**

"Los índices son estructuras optimizadas que MongoDB genera para acelerar las consultas. Sin un índice, cuando buscamos un documento MongoDB recorre toda la colección uno por uno, lo que se llama COLLSCAN o escaneo completo de la colección. Con un índice, va directamente al documento buscado, lo que se llama IXSCAN."

"Una analogía del material de clase es el índice de un libro: sin índice hay que leer todas las páginas, con índice vas directo a la página que necesitás."

"En este trabajo no implementamos índices porque el volumen de datos es pequeño y es un entorno académico. Sin embargo, en un entorno productivo crearíamos índices simples sobre los campos más consultados: cliente_id en ordenes_trabajo para buscar rápido todas las órdenes de un cliente, estado para filtrar órdenes por etapa de fabricación, y email en clientes para autenticación. Usaríamos explain() con executionStats para verificar si las consultas usan índices o hacen escaneos completos."

BLOQUE 8 — Validaciones



"Las validaciones en MongoDB se implementan usando JSON Schema al momento de crear una colección. Sirven para garantizar que los documentos tengan una estructura correcta antes de insertarse, evitando datos inconsistentes, campos faltantes o tipos de datos incorrectos."

"En este trabajo no implementamos validaciones formales, pero sabemos cómo aplicarlas. Por ejemplo, podríamos validar que el campo email tenga formato de correo mediante una expresión regular, que precio_base sea siempre un número mayor a cero, que el campo estado solo pueda contener los valores Pendiente, En Fabricación o Entregado usando enum, y que nombre y email sean campos obligatorios en clientes. MongoDB rechazaría automáticamente cualquier inserción que no cumpla esas reglas."

BLOQUE 9 — Infraestructura Atlas y acceso



"El cluster está desplegado en MongoDB Atlas, que es la plataforma cloud de MongoDB. Usamos el tier gratuito M0 con servidor en AWS São Paulo. Atlas nos provee el cluster como un Replica Set de 3 nodos, lo que garantiza disponibilidad de los datos."

"Para la configuración de seguridad creamos dos usuarios en Database Access: el usuario Damian con permisos de administrador para desarrollo, y el usuario profesor_utm con permisos de solo lectura para que el docente pueda verificar los datos sin poder modificarlos."

"En Network Access configuramos la regla 0.0.0.0/0 que permite conexiones desde cualquier IP. Esto es exclusivamente para fines académicos para facilitar la corrección. En producción real la buena práctica es declarar únicamente las IPs conocidas y autorizadas, como la del servidor de aplicación o del equipo de desarrollo, para minimizar la superficie de ataque."

PREGUNTAS FRECUENTES DEL DOCENTE

Estas preguntas pueden hacérselas a cualquiera. Es importante que ambos sepan responderlas.

? ¿Por qué eligieron MongoDB y no una base relacional como MySQL?

"Porque el modelo de datos del taller es flexible y orientado a documentos. En MongoDB los datos que se acceden juntos se almacenan juntos, lo que hace las consultas más rápidas y simples. Una base relacional requeriría múltiples tablas y

joins complejos para obtener la información completa de una orden. Además, MongoDB permite esquemas flexibles, útil cuando la estructura puede variar."

? ¿Qué es BSON?

"BSON es Binary JSON, el formato interno que usa MongoDB para almacenar los documentos. Es similar a JSON pero en binario, lo que permite mayor eficiencia y soporta tipos de datos adicionales como ObjectId, fechas y números de precisión específica."

? ¿Qué es un ObjectId?

"Es el identificador único que MongoDB genera automáticamente para cada documento. Es un valor de 12 bytes que incluye un timestamp, un identificador de máquina y un contador. Garantiza que no haya duplicados incluso en entornos distribuidos."

? ¿Qué pasa si cambia el precio de un material?

"Actualizamos el precio en la colección materiales con un updateOne y \$set. Las órdenes históricas no se ven afectadas porque tienen los insumos_utilizados embebidos con los valores del momento exacto de la fabricación. Esta es la ventaja del patrón de congelamiento histórico que implementamos."

? ¿Cómo buscarían todas las órdenes de un cliente?

"Con db.ordenes_trabajo.find({ cliente_id: ObjectId('...') }). Filtramos por el campo cliente_id que referencia al documento del cliente. En producción este campo tendría un índice simple para acelerar esa consulta."

? ¿Por qué no embebieron el cliente dentro de la orden?

"Porque la relación es uno a muchos con crecimiento no acotado. Un cliente puede tener cientos de órdenes. Embeber duplicaría nombre, email, teléfono y dirección en cada orden. Con linking actualizamos en un solo punto y evitamos redundancia masiva."

? ¿Qué diferencia hay entre embedding y linking?

"Embedding guarda los datos relacionados dentro del mismo documento, ideal para relaciones uno a uno o uno a pocos donde los datos siempre se consultan juntos. Linking guarda solo el ID del documento relacionado, ideal para relaciones uno a muchos donde los datos crecen indefinidamente o se consultan por separado."

? ¿Qué es la escalabilidad horizontal?

"Es la capacidad de distribuir los datos en múltiples servidores para manejar grandes volúmenes de información. A diferencia de la escalabilidad vertical que mejora el hardware de un único servidor, la horizontal agrega más servidores. MongoDB está diseñado para escalar horizontalmente."

? ¿Qué son las propiedades ACID y las tienen en MongoDB?

"ACID significa Atomicidad, Consistencia, Aislamiento y Durabilidad. Son propiedades que garantizan la integridad de las transacciones. Las bases relacionales las cumplen de forma nativa. MongoDB las soporta a nivel de documento individual siempre, y a partir de versiones recientes también en transacciones multi-documento, aunque esto no es lo habitual en el modelado NoSQL."

? ¿Por qué tienen activo: false en algunos documentos de clientes?

"Porque implementamos baja lógica. El negocio no permite eliminar registros físicamente para preservar el historial de auditoría. En lugar de deleteOne usamos updateOne con \$set activo: false. Esos clientes ya no aparecerán en las consultas operativas del sistema, pero sus datos históricos quedan preservados."

NOTAS FINALES PARA LA DEFENSA

- Tener Compass abierto con las 4 colecciones visibles antes de entrar.
- Damian navega en Compass mientras Ximena habla de los bloques que le corresponden.
- Si preguntan algo que no saben, no inventar. Decir: 'Eso no lo implementamos, pero sabemos que en un entorno productivo se haría con...'
- Si preguntan por validaciones o índices: reconocer que no los implementaron y explicar cómo se harían.
- La palabra clave para embedding: 'siempre se consultan juntos'.
- La palabra clave para linking: 'relación uno a muchos con crecimiento no acotado'.
- La palabra clave para baja lógica: 'updateOne con \$set activo: false en lugar de deleteOne'.